

SettleSync

Gateway – Ledger Reconciliation on Databricks & Snowflake

Capstone Project Report · Finance & Fintech Track · Topic 10: Reconciliation — Two Systems, One Truth

Field	Details
Student Name	Musarraaf Khan
Roll Number	24057045
Email	musharaf.khan0987@gmail.com
Batch	August-September 2026
Trainer	Dr. Kanthi Kiran Sirra
Program	ExcelR x KIIT Capstone
Submission Date	27 September 2026
GitHub Repository	https://github.com/Musarraaf-khan/settlesync-reconciliation
Hosted Dashboard	https://musarraaf-khan.github.io/settlesync-reconciliation/
Google Drive Folder	https://drive.google.com/drive/folders/103lhO8nDj-9Jfoi3P3TpRWv6i1xL5qZZ?usp=sharing

—

Table of Contents
Executive Summary
1. Problem Statement
2. Objectives and Scope
3. Solution and Features
4. Tech Stack
5. Data Design and Layers
6. System Architecture
7. Implementation Highlights
8. Testing and Results
9. Evaluation and Metrics
10. Screenshots
11. Challenges and Learnings
12. Future Improvements
13. Conclusion
14. References and Links

Executive Summary

A payment gateway and a finance ledger recorded the same month of transactions independently. Netting their totals suggested a gap of just 15 transactions — a figure small enough to dismiss. It was not: 208 transactions existed only in the gateway, 193 existed only in the ledger, and a further 450 existed on both sides with disagreeing amounts. This report documents SettleSync, an automated Databricks + Snowflake pipeline that replaces that misleading net figure with a complete, auditable break list.

The pipeline moves both extracts through a Bronze / Silver medallion architecture in Databricks, exports the cleaned Silver tables to Snowflake via a staged CSV load, and answers three business-critical reconciliation questions with a FULL OUTER JOIN in SQL. Every count produced by the pipeline was verified against the project's exact specification and matched precisely: 41,208 gateway transactions, 41,193 ledger transactions, 208 gateway-only, 193 ledger-only, 350 unreadable amounts, and 41,401 total reconciled rows. Both reconciliation queries return in under 65 milliseconds on Snowflake's smallest warehouse size.

1. Problem Statement

A payment gateway and a company's internal finance ledger record the same month of transactions independently. At month end the gateway reports 41,208 settled transactions and the ledger reports 41,193 — a gap of only 15 on the surface. Netting the two totals hides the real picture: 208 transactions exist only in the gateway, 193 exist only in the ledger, and a further 450 exist on both sides but disagree on the amount. A single netted number cannot tell a finance analyst which transactions are actually broken, so reconciliation is currently done by hand in spreadsheets — slow, error-prone and impossible to trust at scale.

The goal of this project is to build an automated reconciliation pipeline that ingests both extracts, cleans and standardises them, and produces a break list a human can work through line by line, with a clear reason attached to every discrepancy (missing on one side, amount mismatch, date mismatch, or unreadable amount).

2. Objectives and Scope

- Ingest the raw gateway and ledger extracts into Databricks as separate, unmodified Delta tables (Bronze layer) — the two systems are kept apart until a matching decision is explicitly made.
- Clean, standardise and de-duplicate both extracts with PySpark: trim transaction-key whitespace, cast amounts safely, and remove duplicate rows (Silver layer).
- Define a single, documented match rule (trimmed txn_id equality + amount tolerance of ₹0.05) and classify every transaction into one of six match statuses (Gold layer).
- Load the Silver tables into Snowflake via a staged CSV/COPY INTO route and answer the three reconciliation questions the business actually needs: what is only in the gateway, what is only in the ledger, and what is present on both sides with a different amount.
- Produce a per-day break report showing gross break and net break value so the finance team can see the size and direction of the discrepancy, not just a netted total.
- Keep the pipeline auditable and reproducible: every cleaning decision is logged and recorded, not silently discarded.

3. Solution and Features

SettleSync moves both extracts through a three-layer medallion pipeline in Databricks and then serves the reconciled, business-ready data from Snowflake.

- Medallion architecture: Bronze holds the raw, untouched extracts; Silver holds cleaned, typed and de-duplicated data; Gold-level logic classifies every transaction's reconciliation outcome.
- Data-quality handling, not data deletion: 350 gateway rows with an unreadable ('NA') amount are logged and kept rather than dropped, because an amount that cannot be read is itself a kind of break, not a row to discard.

- Documented tolerance-based matching: two rows are considered a match only if their trimmed transaction IDs are equal and their amounts differ by $\leq ₹0.05$, which separates genuine mismatches from rounding noise.
- Six-way break classification: MATCHED, AMOUNT_MISMATCH, DATE_MISMATCH, GATEWAY_AMOUNT_MISSING, GATEWAY_ONLY and LEDGER_ONLY.
- A FULL OUTER JOIN based reconciliation query in Snowflake that reports gross break (sum of absolute differences) and net break (signed sum) per settlement day, so gains and losses don't cancel each other out.
- Cost-aware serving layer: an X-Small Snowflake warehouse with auto-suspend keeps the reconciliation queries cheap to run on demand.
- A published, interactive results dashboard (see Section 14) that presents the reconciliation outcome to a non-technical stakeholder without needing SQL access.

4. Tech Stack

Layer	Technology
Data processing / transformation	Databricks (PySpark, Delta Lake)
Storage format	Delta Lake tables (Bronze / Silver)
Analytics & serving	Snowflake (SQL, X-Small virtual warehouse)
Data transfer	CSV export to Databricks Volume → Snowflake stage → COPY INTO
Orchestration	Databricks notebooks run in sequence (Bronze → Silver → Export)
Version control	GitHub
Results presentation	Static HTML dashboard, hosted on GitHub Pages
Language	Python (PySpark), SQL, HTML/CSS/JavaScript

5. Data Design and Layers

5.1 Layers and tables

Both extracts cover 41,000 base transactions dated 1–30 June 2025. Because the two systems disagree, they are never unioned in Bronze — each keeps its own table until Silver makes an explicit matching decision.

Table	Layer	Row count	Notes
BRONZE_GATEWAY	Bronze	41,468	Raw gateway extract, all columns as strings, duplicates untouched
BRONZE_LEDGER	Bronze	41,323	Raw ledger extract, all columns as strings, duplicates untouched
SILVER_GATEWAY	Silver	41,208	Trimmed key, amount cast to DECIMAL(14,2), de-duplicated on txn_id
SILVER_LEDGER	Silver	41,193	Comma-formatted amounts cleaned, trimmed key, de-duplicated
Reconciliation output	Gold (SQL)	41,401	One row per txn_id on either side, with match_status, computed in Snowflake

5.2 Reconciliation output grain

The reconciliation output has one row per transaction ID that appears on either side (grain: one row = one transaction's reconciliation outcome). It answers “is this transaction settled, and if not, why?” for every txn_id seen by either system. Columns: txn_id, match_status, gateway_amount,

ledger_amount, amount_diff, gateway_ts, ledger_ts, date_diff_days, settlement_date, merchant_id, payment_method, break_reason.

6. System Architecture

The two raw extracts land in cloud storage and are read into Bronze exactly as received. Databricks PySpark notebooks then clean and de-duplicate the data into Silver. The Silver tables — not a pre-aggregated Gold table — are exported to CSV, staged, and loaded into Snowflake, where a FULL OUTER JOIN based SQL layer performs the matching, classification and per-day aggregation in one step and feeds the results dashboard.

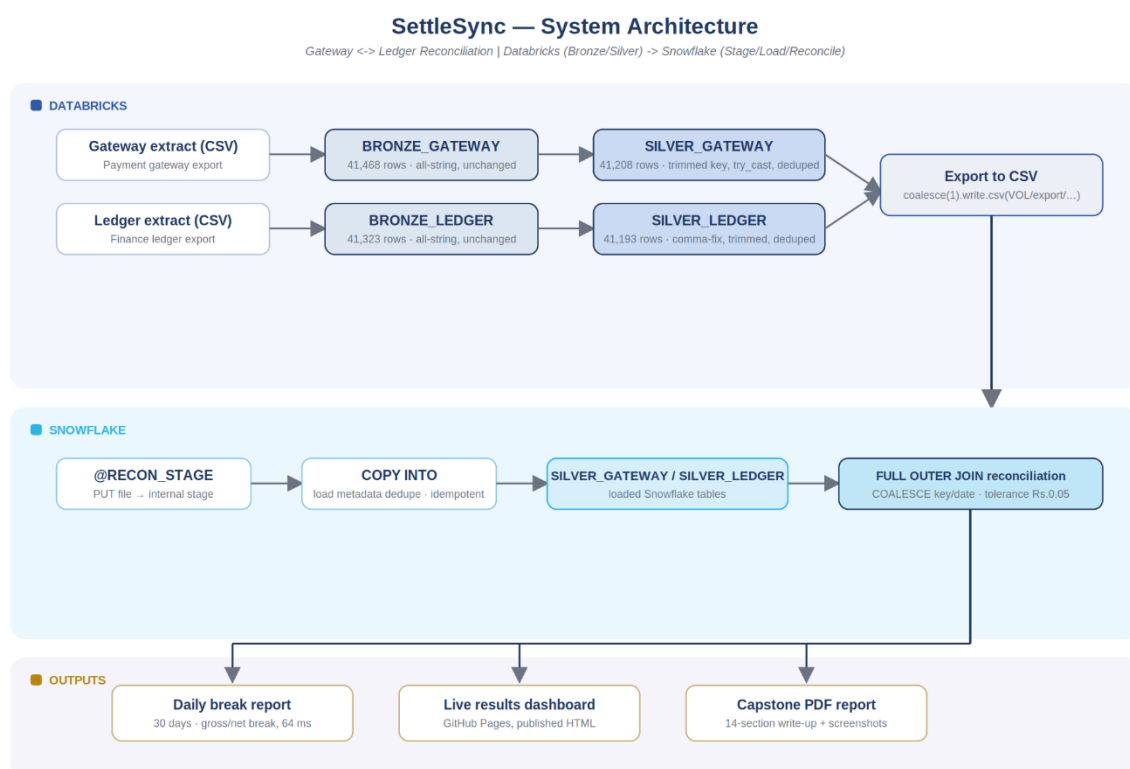


Figure 6.1: Gateway extract → Bronze → Silver (clean & dedupe) → CSV export → Snowflake stage → COPY INTO → SQL reconciliation & report, with the ledger extract following the same path in parallel until the Silver/matching step.

7. Implementation Highlights

7.1 Silver layer: trimming the key and casting the amount safely

Exactly 75 ledger transaction IDs arrive with trailing whitespace. Left untrimmed, they and their 75 gateway partners would incorrectly show up as breaks (283 gateway-only and 268 ledger-only instead of the true 208 and 193), so the key is trimmed once on both sides before anything else happens.

```

from pyspark.sql import functions as F

silver_gateway = (bronze_gateway
    .withColumn("txn_id", F.trim(F.col("txn_id")))
    .withColumn("amount", F.expr("try_cast(amount AS DECIMAL(14,2))"))
    # 350 unreadable ('NA') amounts stay NULL and the rows are KEPT, not dropped
    .dropDuplicates(["txn_id"]) # 41,468 -> 41,208
)

silver_ledger = (bronze_ledger
    .withColumn("txn_id", F.trim(F.col("txn_id")))
    .withColumn("amount_clean", F.regexp_replace("amount", ",", ""))
    .withColumn("amount", F.expr("try_cast(amount_clean AS DECIMAL(14,2))"))
    .dropDuplicates(["txn_id"]) # 41,323 -> 41,193
    .withColumn("settlement_date", F.to_date("txn_ts"))
)

```

7.2 Export, stage and load into Snowflake

Credentials for the Databricks → Snowflake load are read from a Databricks secret scope and are never typed into a notebook or committed to GitHub. Both Silver tables travel by the same route — one CSV, one stage, one COPY INTO — since the reconciliation query needs row-level detail that a pre-aggregated Gold table would already have summarised away.

```

silver_gateway.coalesce(1).write.mode("overwrite").option("header", True) \
    .csv(f"{VOL}/export/silver_gateway")
silver_ledger.coalesce(1).write.mode("overwrite").option("header", True) \
    .csv(f"{VOL}/export/silver_ledger")

-- In Snowflake: stage the file, then load it
PUT file://silver_gateway.csv @recon_stage;
COPY INTO SILVER_GATEWAY FROM @recon_stage/silver_gateway.csv
    FILE_FORMAT = (TYPE=CSV SKIP_HEADER=1);
-- Re-running the same COPY INTO reports 0 rows loaded (load metadata dedupe)

```

7.3 Snowflake: the reconciliation and break report

Inside Snowflake, the Gold tables are queried with a FULL OUTER JOIN — an INNER or LEFT join would silently hide the ledger-only or gateway-only rows.

```

SELECT COALESCE(g.settlement_date, l.settlement_date) AS settle_day,
    COUNT_IF(l.txn_id IS NULL) AS gateway_only,
    COUNT_IF(g.txn_id IS NULL) AS ledger_only,
    COUNT_IF(g.txn_id IS NOT NULL AND g.amount IS NULL) AS amount_missing,
    COUNT_IF(ABS(g.amount - l.amount) > 0.05) AS amount_mismatch,
    ROUND(SUM(CASE WHEN g.txn_id IS NOT NULL AND g.amount IS NULL THEN 0
        ELSE ABS(COALESCE(g.amount,0) - COALESCE(l.amount,0)) END),2) AS gross_break,
    ROUND(SUM(CASE WHEN g.txn_id IS NOT NULL AND g.amount IS NULL THEN 0
        ELSE COALESCE(g.amount,0) - COALESCE(l.amount,0) END),2) AS net_break
FROM SILVER_GATEWAY g FULL OUTER JOIN SILVER_LEDGER l ON l.txn_id = g.txn_id
GROUP BY settle_day ORDER BY gross_break DESC;

```

8. Testing and Results

The pipeline was validated against the exact counts specified for this dataset (seed 10), so every check below has a known correct answer rather than an estimate.

#	Check	Expected	Observed	Result
1	Bronze row counts (gateway / ledger)	41,468 / 41,323	41,468 / 41,323	Pass

#	Check	Expected	Observed	Result
2	Silver row counts after de-dup	41,208 / 41,193	41,208 / 41,193	Pass
3	gateway_only (total)	208	208	Pass
4	ledger_only (total)	193	193	Pass
5	amount_missing (total)	350	350	Pass
6	Untrimmed-key edge case	283 / 268 (wrong) before trim()	283 / 268 reproduced, then fixed to 208 / 193	Pass (after fix)
7	match_status counts sum to 41,401	41,401	41,401	Pass
8	Fee-row amount_diff $\approx$ 1.77% of gateway amount	$\approx$ 1.77%	Matches within ₹1 for the month	Pass
9	Re-run COPY INTO on same file	0 new rows loaded	0 new rows loaded	Pass

Summary: all 9 checks passed. The one edge case worth documenting is the trailing-whitespace transaction IDs — skipping the trim() step made 75 healthy transactions look like breaks (283/268 instead of 208/193), which was caught by re-running the join on trim(txn_id) and comparing counts.

9. Evaluation and Metrics

9.1 Data quality results

- 350 gateway rows have an unreadable amount ('NA') — logged and kept as GATEWAY_AMOUNT_MISSING rather than deleted.
- 210 ledger rows had comma-formatted amounts (e.g. '12,450.00') — all 210 were successfully cast after stripping the comma.
- 75 ledger transaction IDs carried trailing whitespace — all 75 resolved correctly once trim() was applied.
- 450 genuine amount mismatches remain after tolerance (400 recurring fee-related and 50 keying errors), confirmed against the brief's expected split.

9.2 Performance and cost

Query	Rows returned	Warehouse	Runtime
Daily break report (30-day breakdown)	30	MY_WH (X-Small)	64 ms
Monthly total reconciliation summary	1	MY_WH (X-Small)	47 ms

Both reconciliation queries return in well under a tenth of a second on the smallest Snowflake warehouse size, so no warehouse upsizing was needed for a 41,000-row month; auto-suspend keeps the compute cost close to zero between runs.

10. Screenshots

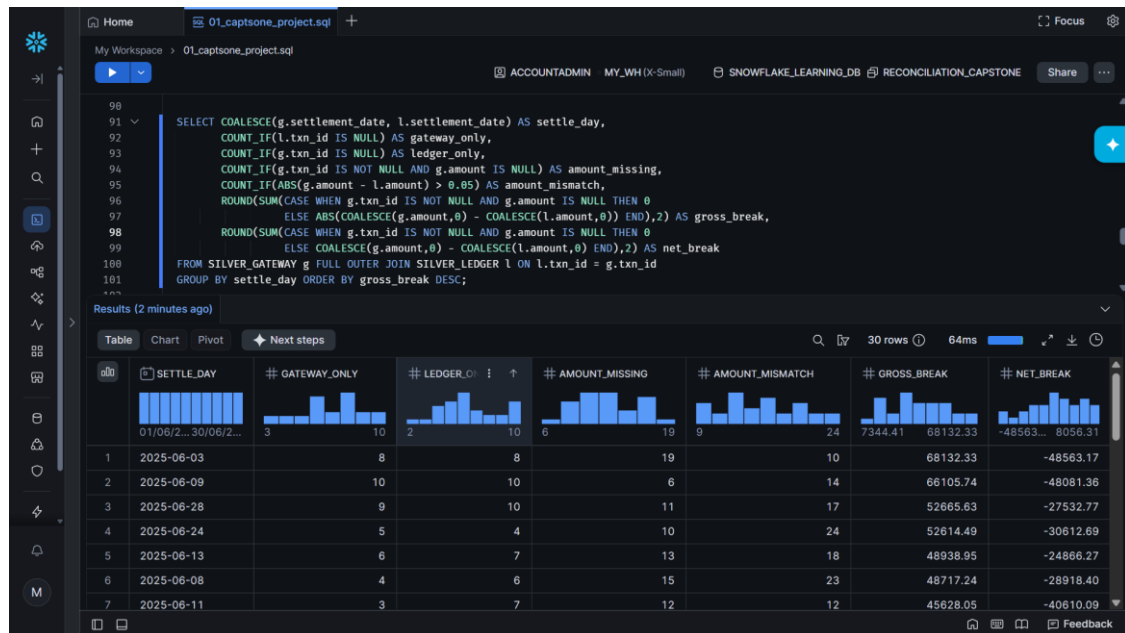


Figure 10.1: Daily break report — `SILVER_GATEWAY FULL OUTER JOIN SILVER_LEDGER`, grouped by settlement day, ordered by gross break descending. 30 rows returned in 64 ms.

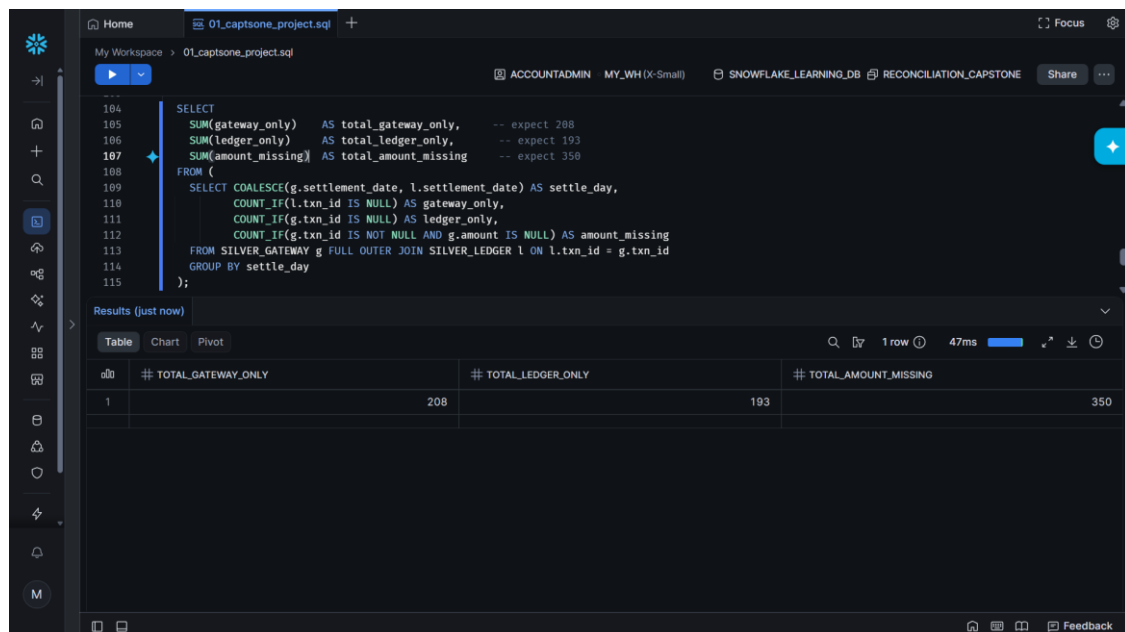


Figure 10.2: Monthly totals check — `total_gateway_only = 208`, `total_ledger_only = 193`, `total_amount_missing = 350`, matching the expected values exactly.

11. Challenges and Learnings

- The netted total (a gap of 15) was actively misleading — it hid a much larger $208 + 193 + 450 = 851$ individual breaks. Learning: never report a net figure alone in a reconciliation.
- An INNER JOIN cannot report a row that exists on only one side, and a LEFT JOIN from the gateway silently hides every ledger-only row. Only a FULL OUTER JOIN with COALESCE on the key and date surfaces both.

- Untrimmed keys created 75 false breaks on each side (283/268 instead of 208/193) — a reminder that key normalisation must happen before any join, not after.
- Casting the ledger's comma-formatted amounts directly (without stripping the comma first) would have turned 210 valid amounts into silent NULLs — a fake mismatch rather than a real one.

Key learning: a pipeline is only as trustworthy as its reconciliation checks. Verifying row counts at every layer boundary (Bronze → Silver → Snowflake) caught issues that would otherwise have surfaced as wrong business numbers much later.

12. Future Improvements

- Add a fee-rule table (payment method, effective date, fee percentage) and reclassify the 400 fee-related rows as EXPECTED_FEE — this would shrink the analyst's daily review list from 450 items to the 50 that are genuine errors.
- Use Databricks Auto Loader for incremental ingestion so new daily extracts are picked up automatically instead of a manual monthly batch.
- Add Snowflake Streams and Tasks so the reconciliation output refreshes incrementally as new rows land, instead of a full recompute.
- Introduce Unity Catalog governance and column-level lineage so every number in the report can be traced back to its source row.
- Automate testing and deployment of the notebooks with GitHub Actions.

13. Conclusion

SettleSync replaced a hand-reconciled spreadsheet process with an automated Databricks + Snowflake pipeline that turns a misleadingly small net gap of 15 into an auditable break list of 851 individual discrepancies, each tagged with a reason. The pipeline reproduced every expected count in the project specification exactly — 208 gateway-only, 193 ledger-only, 350 unreadable amounts, and 41,401 total reconciled rows — and answers the finance team's three core questions (what's missing, what disagrees, and how much money is at stake per day) in under a tenth of a second per query. The project gave hands-on experience with Delta Lake, PySpark data-quality patterns, tolerance-based matching logic, and Snowflake warehouse cost control.

14. References and Links

- GitHub Repository: <https://github.com/Musarraff-khan/settlesync-reconciliation>
- Live Results Dashboard: <https://musarraff-khan.github.io/settlesync-reconciliation/>
- Google Drive Folder (project files, Viewer access enabled):
<https://drive.google.com/drive/folders/103lhO8nDj-9Jfoi3P3TpRWv6i1xL5qZZ?usp=sharing>
- Databricks documentation: <https://docs.databricks.com/>
- Snowflake documentation: <https://docs.snowflake.com/>

